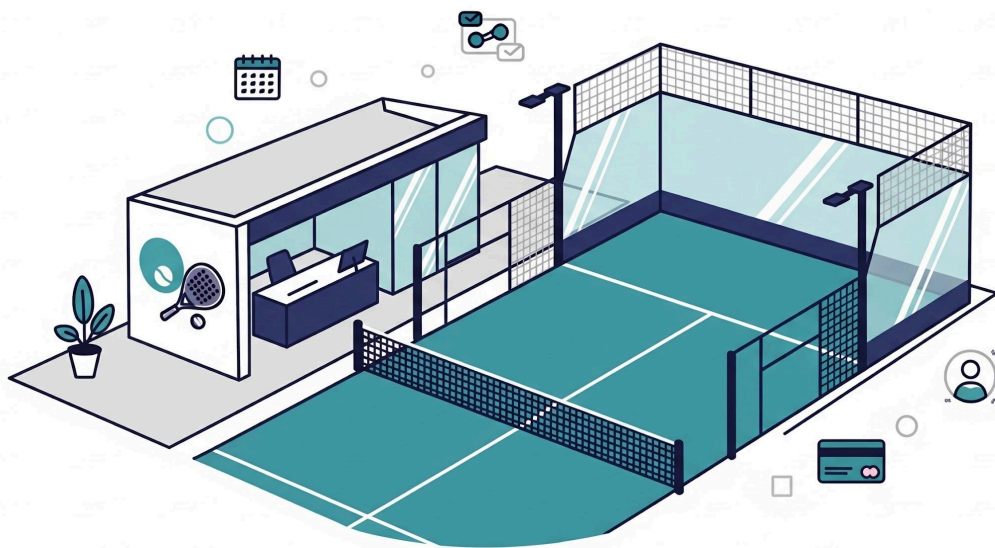




Gestion Terrain de Padel

Cahier des charges

2025 - 2026

—

Aimé Nkurunziza

PSR 13868

Projet SGBD & PDW

V. Fievez & R.Hardenne

Table des matières

Contexte.....	4
Cadre.....	4
Projet.....	4
Interprétation personnelle de l'énoncé.....	4
Analyse Métier.....	8
Description de la solution envisagée.....	8
Les intervenants.....	8
Membre Global (Gxxxx).....	8
Membre Site (Sxxxx).....	8
Membre Libre (Lxxxx).....	8
Administrateur Global.....	8
Administrateur de Site.....	9
Fonctions attendues.....	9
Exigences fonctionnelles - Membre.....	9
Exigences fonctionnelles - Administrateur Global.....	9
Exigences fonctionnelles - Administrateur de Site.....	10
Exigences non fonctionnelles.....	11
Contrainte business.....	11
Gestion des matchs.....	11
Gestion des réservations.....	12
Gestion des membres.....	12
Gestion des sites et terrains.....	12
Analyse Fonctionnelle.....	13
Diagramme Use Cases.....	13
Description Textuelles des Use Cases.....	14
UC-01 Créer un compte.....	14
UC-02 — Se connecter.....	15
UC-03 — Créer un match privé et ajouter des joueurs.....	16
UC-04 — Créer un match public.....	17
UC-05 — S'inscrire à un match public.....	18
UC-06 — Payer sa participation à un match.....	19
UC-07 — Gérer un site (horaires, fermetures, terrains).....	20
UC-08 — Générer les créneaux de disponibilité.....	21
UC-09 — Consulter les statistiques et le chiffre d'affaires.....	22
Contraintes Fonctionnelles.....	23
Contraintes liées aux Matches (CF-M).....	23

Contraintes liées aux Réservations (CF-R).....	25
Contraintes liées aux Membres (CF-MB).....	27
Contraintes liées aux Sites et Terrains (CF-S).....	29
Contraintes de Sécurité (CF-SEC).....	29
Description des entités.....	30
Personne.....	31
TypeMembre.....	31
Membre.....	32
Penalite.....	33
Administrateur.....	33
Site.....	34
Terrain.....	34
HoraireAnnuel.....	35
FermetureRecurrente.....	35
FermeturePonctuelle.....	36
Disponibilite.....	36
MatchPadel.....	37
Participation.....	38
Paieement.....	38
RefreshToken.....	39
Schéma relationnel de la solution.....	40
Schéma relationnel.....	40
Schéma EA.....	41
Analyse technique.....	42
Stack technologique.....	42
Architecture Backend - Spring Boot.....	43
Controllors — @RestController.....	43
Services — @Service.....	43
Repositories — JpaRepository.....	43
Models — @Entity.....	44
DTOs — Data Transfer Objects.....	44
Injection de dépendances.....	44
Architecture Frontend - Angular.....	44
Base de données - PostgreSQL.....	45
Justification du choix PostgreSQL.....	45
Démarrage via Docker Compose.....	45
Initialisation automatique — Flyway.....	45
Sécurité — Utilisateurs SQL.....	46
Choix d'accès aux données.....	46

Gestion terrain de padel

Authentification — JWT.....	47
Flux d'authentification.....	47
Sécurité des mots de passe.....	48
Génération des Créneaux — Approche Précalculée.....	48
Algorithme de génération.....	49
Traitements Automatiques — Spring Scheduler.....	50
Tests.....	51
Tests unitaires — Mockito.....	51
Tests d'intégration — Testcontainers.....	51
Résumé des Choix Techniques.....	52
Annexes.....	53
Enoncé Examen.....	53

Contexte

Cadre

Ce projet est réalisé dans le cadre du cours de Projet de développement SGBD (2025-2026) à l'EPHEC, sous la direction du professeur V. Fievez. Il s'inscrit également dans le cadre du cours de Projet de développement Web (PDW), dispensé par le professeur R. Hardenne.

L'objectif est de démontrer l'acquisition des compétences enseignées dans ces deux cours : modélisation de bases de données relationnelles, développement d'une API REST, et mise en place d'une application web complète avec séparation frontend / backend.

Le projet est réalisé individuellement.

Projet

Le projet consiste en la conception et le développement d'une application web de gestion de terrains de padel, nommée "Padel Manager". Cette application permet à un gestionnaire de gérer plusieurs sites, chacun disposant d'un nombre variable de terrains, avec des horaires et des jours de fermeture propres à chaque site.

L'application doit permettre aux membres de réserver des créneaux de jeu (matchs privés ou publics), de gérer les paiements associés, et aux administrateurs de superviser l'activité globale ou par site.

Le projet se décompose en trois livrables :

- Un cahier des charges (analyse complète)
- Une base de données relationnelle
- Une application web : frontend Angular et backend Spring Boot (REST API)

Interprétation personnelle de l'énoncé

L'énoncé original présentant certaines ambiguïtés, les interprétations suivantes ont été retenues et sont assumées tout au long de ce cahier des charges.

Hiérarchie des types de membres

L'énoncé distingue trois types de membres avec des règles de réservation différentes. L'interprétation suivante a été retenue afin de créer une hiérarchie cohérente et défendable :

Gestion terrain de padel

Type	Matricule	Créer un match	S'inscrire (match public)
Membre Global	Gxxxx	jusqu'à 21 jours avant	dès la création
Membre Site	Sxxxx	jusqu'à 14 jours avant (son site uniquement)	dès la création
Membre Libre	Lxxxx	interdit	dans les 5 derniers jours seulement

Justification : le terme "réservation" dans l'énoncé désigne la création d'un match pour les membres Global et Site. Le membre Libre, n'ayant pas d'engagement envers un site, ne peut pas créer de match mais peut s'inscrire à des matchs publics existants dans les 5 derniers jours avant le début. Cette interprétation crée une vraie distinction entre les trois types et justifie l'existence de chaque statut.

Invitation dans un match privé

L'organisateur ajoute les autres joueurs en encodant leur matricule directement dans l'application. Le système vérifie l'existence du matricule et retourne un message d'erreur si celui-ci est inconnu. L'organisateur est responsable de prévenir ses joueurs par ses propres moyens (ex: WhatsApp, SMS). Le joueur ajouté découvre sa participation en se connectant avec son matricule et voit le match dans son tableau de bord avec le statut "en attente de paiement".

Pénalité et solde cumulés

En cas de match privé non complet la veille (24h avant le début), le match bascule en public et une pénalité de 7 jours est appliquée à l'organisateur (impossibilité de créer un nouveau match). Si le jour J des places restent non remplies, l'organisateur doit également s'acquitter du solde correspondant aux places vides (nombre de places vides × 15€).

Les deux sanctions sont cumulables.

Annulation de match

Les délais d'annulation suivants ont été retenus sur base des notes de cours :

- Match privé : annulation possible jusqu'à 48h avant le match
- Match public : annulation possible jusqu'à 24h avant le match
- Seul l'organisateur peut annuler le match

Authentification

L'énoncé SGBD mentionne "pas de login, uniquement le matricule". Cependant, les exigences du cours PDW imposent un mécanisme d'authentification avec gestion des rôles. L'interprétation retenue est la suivante : l'authentification se fait via matricule + mot de passe, stockés en base de données. Le système génère un JWT token qui permet d'identifier le rôle de l'utilisateur (MEMBRE_GLOBAL, MEMBRE_SITE, MEMBRE_LIBRE, ADMIN_SITE, ADMIN_GLOBAL).

Cette approche satisfait les deux cours.

Inscription des membres

L'inscription est libre et immédiate. Tout utilisateur peut créer un compte en choisissant son type de membre. Si le type Site est choisi, il doit sélectionner son site dans la liste disponible. Le matricule est généré automatiquement par le système selon le format correspondant au type choisi. Aucune validation par un administrateur n'est requise.

Les comptes administrateurs sont créés directement en base de données par l'administrateur global. Il n'existe pas d'interface d'inscription pour les administrateurs.

Cumul des comptes

Une même personne peut posséder au maximum deux comptes : un compte Global (G) et un compte Site (S). Le compte Libre (L) est exclusif et ne peut pas être cumulé avec un autre type. Les combinaisons autorisées sont donc : G seul, S seul, L seul, ou G+S. Un seul compte S est autorisé par personne.

En cas de blocage (pénalité, solde dû), les deux comptes G et S de la même personne sont bloqués simultanément. Le paiement d'un solde dû via l'un des deux comptes remet le solde à zéro pour les deux. Les vérifications de blocage se font donc au niveau de la personne (id_personne) et non du matricule.

La connexion se fait toujours via un matricule spécifique. Si une personne possède deux comptes, elle doit se connecter séparément avec chaque matricule selon le contexte souhaité.

Gestion des paiements

Le paiement est simulé en base de données. Chaque joueur paye sa part individuelle de 15€ (60€ / 4 joueurs). Le paiement valide la participation au match. L'organisateur d'un match public est responsable du solde en cas de places non remplies.

Gestion des fermetures

La gestion des fermetures est répartie entre l'admin global (fermetures s'appliquant à tous les sites) et l'admin de site (fermetures propres à son site). Deux types de fermetures sont distingués :

→ les fermetures récurrentes (jour de la semaine)

Gestion terrain de padel

→ les fermetures ponctuelles (date précise).

Les horaires d'ouverture sont également gérés par l'admin de site pour son propre site.

Génération des disponibilités

Les créneaux disponibles sont pré-générés à partir des horaires d'ouverture de chaque site (annuels) et des jours de fermeture (globaux ou par site). Chaque créneau a une durée de 1h30 avec 15 minutes d'intervalle entre deux créneaux consécutifs.

Cette approche simplifie la gestion des réservations et la vérification des disponibilités.

Interface unique avec vues par rôle

Une seule application Angular est développée avec des vues différentes selon le rôle de l'utilisateur connecté. La navigation et les fonctionnalités accessibles sont filtrées via des route guards Angular en fonction du rôle retourné par le backend. Cette approche est plus simple à maintenir qu'une application séparée et correspond aux pratiques standard Angular.

Analyse Métier

Description de la solution envisagée

La solution "Padel Manager" est une application web permettant la gestion complète de terrains de padel sur plusieurs sites. Elle offre deux espaces distincts selon le profil de l'utilisateur connecté : un espace membre pour la réservation et la gestion des matchs, et un espace administrateur pour la supervision globale ou par site.

Les membres peuvent créer des matchs privés ou publics, gérer leurs paiements et consulter leurs réservations. Les administrateurs peuvent configurer les sites, les terrains, les horaires, les jours de fermeture et consulter les statistiques d'activité.

Les intervenants

Membre Global (Gxxxx)

Le membre global est un membre premium pouvant réserver sur n'importe quel site. Il peut créer des matchs privés ou publics jusqu'à 21 jours avant la date du match. Il peut s'inscrire à des matchs publics existants sans restriction de site.

Membre Site (Sxxxx)

Le membre de site est rattaché à un site spécifique. Il peut créer des matchs uniquement sur son site jusqu'à 14 jours avant la date du match. Il peut s'inscrire à des matchs publics sur son site dès leur création. Il peut participer comme joueur sur d'autres sites s'il est ajouté par un organisateur.

Membre Libre (Lxxxx)

Le membre libre n'a pas de site attribué. Il ne peut pas créer de match mais peut s'inscrire à des matchs publics sur n'importe quel site, uniquement dans les 5 derniers jours avant la date du match. Il peut également participer comme joueur s'il est ajouté par un organisateur.

Administrateur Global

L'administrateur global a accès à l'ensemble des sites et fonctionnalités d'administration. Il est le seul à pouvoir créer des sites, générer les créneaux de disponibilité, créer des comptes administrateurs et consulter les statistiques globales.

Administrateur de Site

L'administrateur de site ne peut gérer que le site auquel il est rattaché. Il peut consulter les matchs, les réservations, les membres et les statistiques de son site uniquement.

Fonctions attendues

Exigences fonctionnelles - Membre

Code	Description
EF-M-001	S'inscrire en choisissant son type de membre (Global, Site, Libre) et son site si applicable
EF-M-002	Se connecter via matricule et mot de passe
EF-M-003	Consulter son tableau de bord : matchs à venir, matchs en attente de paiement, solde dû
EF-M-004	Créer un match privé (Membres G et S uniquement) en respectant le délai selon son type
EF-M-005	Ajouter des joueurs à un match privé via leur matricule
EF-M-006	Créer un match public (Membres G et S uniquement) en respectant le délai selon son type
EF-M-007	S'inscrire à un match public existant selon les règles de son type de membre
EF-M-008	Payer sa participation à un match (15€ par joueur)
EF-M-009	Annuler un match dont on est organisateur (dans les délais autorisés)
EF-M-010	Consulter la liste des matchs publics disponibles filtrés par site ou globalement
EF-M-011	Consulter l'historique de ses réservations et paiements

Exigences fonctionnelles - Administrateur Global

Gestion terrain de padel

Code	Description
EF-AG-001	Créer et gérer les sites (nom, adresse, ville)
EF-AG-002	Créer et gérer les terrains de tous les sites
EF-AG-003	Consulter et modifier les horaires d'ouverture de tous les sites par année civile
EF-AG-004	Gérer les fermetures globales récurrentes (jour de semaine) s'appliquant à tous les sites
EF-AG-005	Gérer les fermetures globales ponctuelles (date précise) s'appliquant à tous les sites
EF-AG-006	Générer les créneaux de disponibilité pour un site et une année donnés
EF-AG-007	Créer des comptes administrateurs (global ou par site)
EF-AG-008	Consulter l'état des matchs et des terrains sur tous les sites
EF-AG-009	Consulter le chiffre d'affaires global et par site
EF-AG-010	Consulter la liste de tous les membres et leurs informations
EF-AG-011	Consulter les statistiques globales (taux d'occupation, nombre de matchs, revenus par période)

Exigences fonctionnelles - Administrateur de Site

Code	Description
EF-AS-001	Consulter l'état des matchs et des terrains de son site
EF-AS-002	Consulter le chiffre d'affaires de son site
EF-AS-003	Consulter la liste des membres rattachés à son site
EF-AS-004	Consulter les statistiques de son site (taux d'occupation, nombre de matchs, revenus)
EF-AS-005	Gérer les horaires d'ouverture de son site par année civile
EF-AS-006	Gérer les fermetures récurrentes de son site (jour de semaine, ex : fermé tous les lundis)

Gestion terrain de padel

EF-AS-007	Gérer les fermetures ponctuelles de son site (date précise, ex : travaux)
EF-AS-008	Gérer les terrains de son site (ajouter, modifier, désactiver)
EF-AS-009	Générer les créneaux de disponibilité pour son site et une année donnée

Exigences non fonctionnelles

Code	Description
ENF-001	Séparation stricte frontend (Angular 20+) et backend (Spring Boot 3.5+) via REST API
ENF-002	Authentification par JWT avec gestion des rôles (route guards Angular + Spring Security)
ENF-003	Base de données relationnelle PostgreSQL avec un maximum de relations entre les tables
ENF-004	Architecture backend en couches : Controller, Service, Repository, Model
ENF-005	Tests obligatoires sur la couche backend (Controller, Service, Repository)
ENF-006	Utilisation de Git avec issues et branches par fonctionnalité
ENF-007	Dossier d'architecture et document d'exploitation disponibles à la racine du dépôt Git
ENF-008	Sécurité base de données : utilisateurs SQL avec droits spécifiques (pas de superuser)

Contrainte business

Gestion des matchs

- Un match nécessite exactement 4 joueurs pour être validé
- Chaque match coûte 60€ divisé en 4 parts égales de 15€ par joueur
- Le paiement est obligatoire et valide la participation au match
- Un match privé non complet la veille (24h avant le début) devient automatiquement public
- Une pénalité de 7 jours est appliquée à l'organisateur d'un match privé non complet
- L'organisateur d'un match public paye le solde des places non remplies après le jour J
- Les deux sanctions (pénalité + solde) sont cumulables

Gestion terrain de padel

- Un organisateur avec un solde dû le voit ajouté automatiquement à son prochain paiement
- Un organisateur en période de pénalité ne peut pas créer de nouveau match

Gestion des réservations

- Les créneaux ont une durée fixe de 1h30 avec 15 minutes d'intervalle entre deux créneaux
- Les horaires sont définis par site et valables pour une année civile entière
- Les jours de fermeture peuvent être globaux (tous sites) ou spécifiques à un site
- Les fermetures peuvent être récurrentes (jour de semaine) ou ponctuelles (date précise)
- Les créneaux disponibles sont pré-générés en tenant compte des horaires et des fermetures
- Un créneau déjà réservé ne peut pas être attribué à un autre match

Gestion des membres

- Un membre peut avoir deux matricules simultanément (ex : Gxxxx + Sxxxx)
- Le matricule est généré automatiquement à l'inscription selon le type choisi
- Un membre en période de pénalité ne peut pas créer de nouveau match
- Un membre avec un solde dû le voit ajouté automatiquement à son prochain paiement
- Le membre de site ne peut créer un match que sur son site d'appartenance
- Le membre libre ne peut pas créer de match mais peut être ajouté comme participant

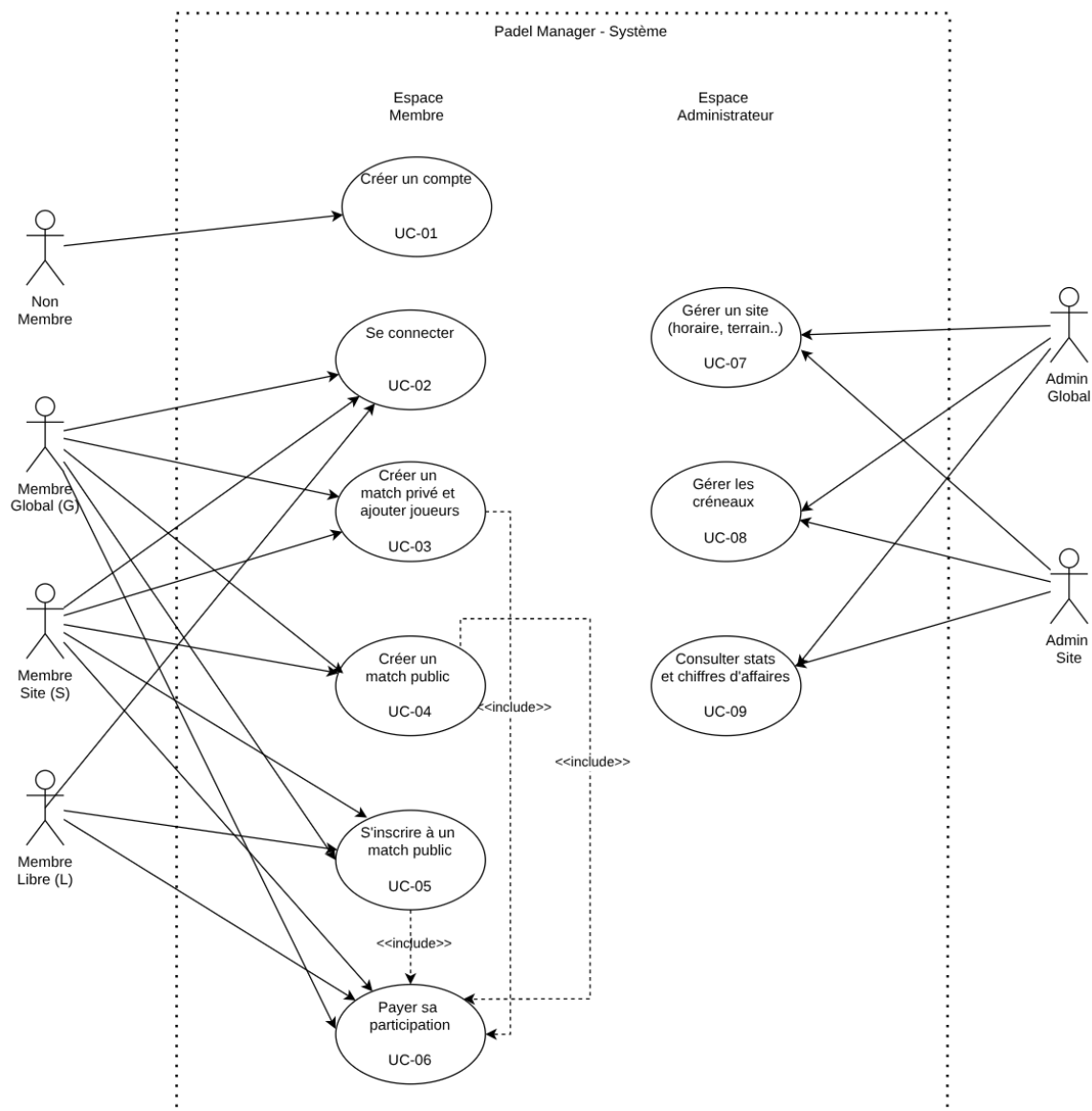
Gestion des sites et terrains

- Chaque site possède un nombre variable de terrains
- Les horaires d'ouverture sont propres à chaque site et définis par année civile
- Seul l'admin global peut créer un nouveau site
- L'admin de site et l'admin global peuvent gérer les terrains d'un site
- La génération des créneaux doit être relancée après toute modification des horaires ou des fermetures

Analyse Fonctionnelle

Diagramme Use Cases

Le diagramme ci-dessous représente les principaux cas d'utilisation de l'application Padel Manager, regroupés par acteur. Seuls les use cases les plus importants sont représentés.



Description Textuelles des Use Cases

UC-01 Créer un compte

Code	UC-01
Titre	Créer un compte
Acteurs	Tout utilisateur non membre
Exigences liées	EF-M-001
Priorité	Haute — prérequis à la connexion et toutes les autres fonctionnalités

Description: Ce cas d'utilisation permet à un utilisateur non inscrit de créer un compte membre en choisissant son type et en fournissant ses informations personnelles.

Déroulement:

1. L'utilisateur accède à la page d'inscription.
2. Il choisit son type de membre : Global (G), Site (S) ou Libre (L).
3. Si le type Site est sélectionné, une liste déroulante affiche les sites disponibles et l'utilisateur en choisit un.
4. Il remplit le formulaire : nom, prénom, email, mot de passe.
5. Il valide l'inscription.
6. Le système génère automatiquement le matricule selon le format du type choisi (Gxxxx / Sxxxx / Lxxxx).
7. Le compte est actif immédiatement. L'utilisateur est redirigé vers la page de connexion.

Cas alternatifs:

- Combinaison email + type de membre déjà utilisé → message d'erreur, l'utilisateur reste sur la page d'inscription.
- Site non sélectionné pour un membre de type S → validation bloquée avec message d'erreur.
- Mot de passe trop court ou invalide → message d'erreur de validation.

Liens avec les contraintes: Un membre peut créer deux comptes distincts avec la même adresse email pour obtenir deux matricules différents (ex : G + S).

UC-02 — Se connecter

Code	UC-02
Titre	Se connecter
Acteurs	Membre Global (G), Membre de Site (S), Membre Libre (L), Administrateur Global, Administrateur de Site
Exigences liées	EF-M-002
Priorité	Haute — prérequis à toutes les fonctionnalités authentifiées

Description: Ce cas d'utilisation permet à tout utilisateur possédant un compte de s'authentifier et d'accéder à son espace selon son rôle.

Déroulement:

1. L'utilisateur accède à la page de connexion.
2. Il saisit son matricule et son mot de passe.
3. Le système vérifie les credentials en base de données.
4. Le système génère un JWT token contenant le rôle de l'utilisateur.
5. L'utilisateur est redirigé vers son tableau de bord selon son rôle (espace membre ou espace administrateur).

Cas alternatifs:

- Matricule ou mot de passe incorrect → message d'erreur, l'utilisateur reste sur la page de connexion.
- Compte inexistant → message d'erreur générique (sans préciser si c'est le matricule ou le mot de passe qui est incorrect, pour des raisons de sécurité).

Liens avec les contraintes: [ENF-002](#) (authentification JWT avec gestion des rôles).

UC-03 — Créer un match privé et ajouter des joueurs

Code	UC-03
Titre	Créer un match privé et ajouter des joueurs
Acteurs	Membre Global (G), Membre de Site (S)
Exigences liées	EF-M-004 , EF-M-005
Priorité	Haute — fonctionnalité centrale de l'application

Description: Ce cas d'utilisation permet à un membre G ou S de créer un match privé sur un créneau disponible et d'y ajouter jusqu'à 3 autres joueurs via leur matricule.

Déroulement:

1. Le membre connecté accède à la section "Créer un match".
2. Il sélectionne le type de match : Privé.
3. Il sélectionne le site (membre G : tous les sites, membre S : son site uniquement).
4. Le système affiche les créneaux disponibles en respectant le délai autorisé (21j pour G, 14j pour S).
5. Le membre sélectionne un créneau disponible.
6. Le système vérifie que le membre n'est pas en période de pénalité et n'a pas de solde dû bloquant.
7. Le match privé est créé avec le statut "en attente de joueurs". L'organisateur est automatiquement inscrit comme premier joueur.
8. L'organisateur saisit les matricules des 3 autres joueurs un par un.
9. Pour chaque matricule saisi, le système vérifie son existence. Si le matricule est inconnu, un message d'erreur est affiché pour ce joueur uniquement.
10. Les joueurs valides sont ajoutés au match avec le statut "en attente de paiement".
11. L'organisateur est redirigé vers la page de détail du match.

Cas alternatifs:

- Aucun créneau disponible pour le site et la période sélectionnée → message informatif.
- Membre en période de pénalité → création bloquée, message indiquant la date de fin de pénalité.
- Matricule d'un joueur inexistant → message d'erreur pour ce joueur, les autres sont quand même ajoutés.
- La veille (24h avant début) du match, si moins de 4 joueurs ont payé → le match bascule automatiquement en public (traitement automatique).

Liens avec les contraintes: [CF-M-001](#) (4 joueurs requis), [CF-M-004](#) (bascule automatique), [CF-M-005](#) (pénalité 7 jours), [CF-MB-003](#) (pénalité bloquante).

UC-04 — Créer un match public

Code	UC-04
Titre	Créer un match public
Acteurs	Membre Global (G), Membre de Site (S)
Exigences liées	EF-M-006
Priorité	Haute — fonctionnalité centrale de l'application

Description: Ce cas d'utilisation permet à un membre G ou S de créer un match public sur un créneau disponible. Les 3 autres places sont ouvertes à tous les membres éligibles.

Déroulement:

1. Le membre connecté accède à la section "Créer un match".
2. Il sélectionne le type de match : Public.
3. Il sélectionne le site selon les restrictions de son type (G : tous sites, S : son site uniquement).
4. Le système affiche les créneaux disponibles en respectant le délai autorisé (21j pour G, 14j pour S).
5. Le membre sélectionne un créneau disponible.
6. Le système vérifie que le membre n'est pas en période de pénalité et n'a pas de solde dû bloquant.
7. Le match public est créé avec le statut "ouvert". L'organisateur est inscrit comme premier joueur avec le statut "en attente de paiement".
8. Le match est immédiatement visible par tous les membres éligibles dans la liste des matchs publics.
9. L'organisateur est redirigé vers la page de détail du match.

Cas alternatifs:

- Membre en période de pénalité → création bloquée, message indiquant la date de fin de pénalité.
- Si le jour J des places sont encore vides → l'organisateur doit payer le solde (places vides × 15€), ajouté à son prochain paiement.

Liens avec les contraintes: [CF-M-001](#) (4 joueurs requis), [CF-M-007](#) (solde organisateur), [CF-MB-003](#) (pénalité bloquante).

UC-05 — S'inscrire à un match public

Code	UC-05
Titre	S'inscrire à un match public existant
Acteurs	Membre Global (G), Membre de Site (S), Membre Libre (L)
Exigences liées	EF-M-007
Priorité	Haute — fonctionnalité centrale de l'application

Description: Ce cas d'utilisation permet à un membre de rejoindre un match public existant selon les règles propres à son type.

Déroulement:

1. Le membre connecté accède à la liste des matchs publics disponibles.
2. Il peut filtrer par site ou consulter tous les matchs selon son type.
3. Le système affiche uniquement les matchs auxquels le membre est éligible:
 - Membre G : tous les matchs publics avec des places disponibles.
 - Membre S : les matchs publics de son site avec des places disponibles.
 - Membre L : uniquement les matchs publics dans les 5 derniers jours avant la date du match.
4. Le membre sélectionne un match et clique sur "Rejoindre ce match".
5. Le système vérifie qu'une place est encore disponible (max 4 joueurs).
6. Le membre est ajouté avec le statut "en attente de paiement" et redirigé vers le paiement.

Cas alternatifs:

- Plus de place disponible → message d'erreur.
- Membre L tentant de s'inscrire à un match à plus de 5 jours → le match n'est pas visible.
- Membre S tentant d'accéder à un match d'un autre site → le match n'est pas visible.

Liens avec les contraintes: [CF-M-001](#) (max 4 joueurs), [CF-MB-001](#) (restrictions par type), [CF-MB-005](#) (membre S limité à son site).

UC-06 — Payer sa participation à un match

Code	UC-06
Titre	Payer sa participation à un match
Acteurs	Membre Global (G), Membre de Site (S), Membre Libre (L)
Exigences liées	EF-M-008
Priorité	Haute — valide la participation au match

Description: Ce cas d'utilisation permet à un membre de payer sa part (15€) pour confirmer sa participation. Si un solde dû est en attente, il est ajouté au montant total.

Déroulement:

1. Le membre accède à son tableau de bord et voit les matchs "en attente de paiement".
2. Il sélectionne le match pour lequel il souhaite payer.
 - Le système affiche le récapitulatif :
 - Part du match : 15€
3. Solde dû éventuel (affiché séparément et ajouté au total)
4. Le membre confirme le paiement.
5. Le système enregistre le paiement avec la date et le montant.
6. Le statut de la participation passe à "confirmée".
7. Si un solde dû existait, il est remis à zéro après le paiement.

Cas alternatifs:

- Le membre annule → il reste en statut "en attente de paiement".
- La veille (24h avant début) du match, si le paiement n'est pas effectué → la place est libérée et le match devient public si c'était un match privé.
- Organisateur d'un match public avec solde dû → le solde est obligatoirement inclus dans le paiement.

Liens avec les contraintes: [CF-M-002](#) (15€ par joueur), [CF-M-003](#) (paiement obligatoire), [CF-MB-004](#) (solde ajouté au prochain paiement).

UC-07 — Gérer un site (horaires, fermetures, terrains)

Code	UC-07
Titre	Gérer un site
Acteurs	Administrateur Global, Administrateur de Site
Exigences liées	EF-AG-002 , EF-AG-003 , EF-AG-004 , EF-AG-005 , EF-AS-005 , EF-AS-006 , EF-AS-007 , EF-AS-008
Priorité	Haute — prérequis à la génération des créneaux

Description: Ce cas d'utilisation permet aux administrateurs de configurer un site : définir ses horaires d'ouverture, ses jours de fermeture et gérer ses terrains.

Déroulement — Gestion des horaires :

1. L'administrateur accède à la section "Gestion du site".
2. Il sélectionne l'année civile.
3. Il saisit l'heure d'ouverture et l'heure de fermeture du site.
4. Il valide. Le système avertit que les créneaux doivent être régénérés.

Déroulement — Gestion des fermetures récurrentes :

1. L'administrateur accède à la section "Fermetures récurrentes".
2. Il sélectionne le(s) jour(s) de la semaine fermés (ex : lundi).
3. Il valide. Le système enregistre les jours de fermeture hebdomadaire.

Déroulement — Gestion des fermetures ponctuelles :

1. L'administrateur accède à la section "Fermetures ponctuelles".
2. Il saisit une date et un motif (ex : travaux, jour férié).
3. Il valide. Le système enregistre la fermeture.

Déroulement — Gestion des terrains :

1. L'administrateur accède à la section "Terrains".
2. Il peut ajouter un terrain (numéro), modifier ou désactiver un terrain existant.
3. Un terrain désactivé n'apparaît plus dans les créneaux disponibles.

Cas alternatifs:

- Heure de fermeture antérieure à l'heure d'ouverture → validation bloquée.
- Modification après génération des créneaux → avertissement de régénération nécessaire.
- Admin de site tentant de modifier un autre site → accès refusé.

Liens avec les contraintes: [CF-S-001](#) (seul l'admin global crée un site), [CF-S-002](#) (admin de site et global gèrent les terrains).

UC-08 — Générer les créneaux de disponibilité

Code	UC-08
Titre	Générer les créneaux de disponibilité
Acteurs	Administrateur Global, Administrateur de Site
Exigences liées	EF-AG-006 , EF-AS-009
Priorité	Haute — prérequis à toute réservation

Description: Ce cas d'utilisation permet de générer automatiquement tous les créneaux disponibles pour un site et une année civile donnés, en tenant compte des horaires et des fermetures.

Déroulement:

1. L'administrateur accède à la section "Génération des créneaux".
2. Il sélectionne le site (admin global : tous les sites, admin de site : son site uniquement).
3. Il sélectionne l'année civile.
4. Le système affiche un récapitulatif : horaires configurés, fermetures détectées, estimation du nombre de créneaux.
5. L'administrateur confirme la génération.
6. Le système génère tous les créneaux selon la logique suivante :
 - Pour chaque jour de l'année civile sélectionnée
 - Si le jour n'est pas une fermeture récurrente ni une fermeture ponctuelle (globale ou du site)
 - Créer les créneaux de 1h30 de l'heure d'ouverture à l'heure de fermeture avec 15 min d'intervalle
7. Le système affiche le nombre de créneaux générés et signale les conflits éventuels.

Cas alternatifs:

- Horaires non configurés → génération bloquée avec message d'erreur.
- Créneaux déjà existants → confirmation avant d'écraser les créneaux libres (les réservés ne sont pas supprimés).
- Admin de site sélectionnant un autre site → accès refusé.

Liens avec les contraintes : CF-Réervations-001 (1h30 + 15 min), CF-Réervations-002 (horaires annuels), CF-Réervations-003 (fermetures globales et par site).

UC-09 — Consulter les statistiques et le chiffre d'affaires

Code	UC-09
Titre	Consulter les statistiques et le chiffre d'affaires
Acteurs	Administrateur Global, Administrateur de Site
Exigences liées	EF-AG-008 , EF-AG-009 , EF-AG-010 , EF-AG-011 , EF-AS-001 , EF-AS-002 , EF-AS-004
Priorité	Moyenne — supervision de l'activité

Description: Ce cas d'utilisation permet aux administrateurs de consulter les statistiques d'activité et le chiffre d'affaires, globalement (admin global) ou par site (admin de site).

Déroulement:

1. L'administrateur accède à la section "Statistiques".
2. Il sélectionne la période souhaitée (semaine, mois, année).
3. L'admin global peut filtrer par site ou consulter les données globales.
4. Le système affiche :
 - Chiffre d'affaires total sur la période (somme des paiements confirmés)
 - Nombre de matchs joués (privés / publics)
 - Taux d'occupation des terrains (créneaux réservés / créneaux disponibles)
 - Nombre de membres actifs sur la période
 - Nombre de matchs annulés
5. L'admin global peut en plus consulter la liste complète des membres.

Cas alternatifs:

- Aucune donnée pour la période → affichage avec valeurs à zéro.
- Admin de site tentant de consulter un autre site → accès refusé.

Liens avec les contraintes: [CF-M-002](#) (15€ par joueur pour calcul CA), [CF-S-002](#) (admin de site limité à son site).

Contraintes Fonctionnelles

Les contraintes fonctionnelles définissent les règles métier qui doivent être respectées par le système. Chaque contrainte est codifiée et son implémentation est décrite aux trois niveaux: base de données (**DB**), couche service (**Service**) et couche contrôleur (**Controller**).

DB: Contraintes, triggers, structure	Service: Logique métier Spring Boot	Controller: Endpoint REST Angular
---	---	---

Contraintes liées aux Matches (CF-M)

Code	Description / Implémentation
CF-M-001	Un match nécessite exactement 4 joueurs pour être joué. Ni plus, ni moins.
	DB : Contrainte CHECK sur la table Participation (max 4 lignes par match_id). Trigger TR_Match_CheckNbJoueurs vérifiant le count avant insertion. Service : MatchService.verifierNbJoueurs(matchId) — lève une exception si count > 4. Controller : MatchController.ajouterJoueur() — retourne HTTP 400 si la règle est violée.
CF-M-002	Chaque match coûte 60€ divisé en 4 parts égales de 15€ par joueur. Le montant par joueur est fixe.
	DB : Colonne montant dans la table Paiement avec valeur par défaut 15.00. Contrainte CHECK montant >= 15.00. Service : PaiementService.calculerMontant() — retourne 15.00 + solde_du éventuel du membre. Controller : PaiementController.initierPaiement() — pré-remplit le montant, non modifiable par l'utilisateur.
CF-M-003	Le paiement est obligatoire pour valider la participation. Sans paiement confirmé, la participation reste en attente.
	DB : Colonne statut dans Participation avec valeur ENUM (EN_ATTENTE, CONFIRMEE, ANNULEE). Contrainte FK vers Paiement. Service : PaiementService.confirmerPaiement() — met à jour le statut de la participation à CONFIRMEE. Controller : PaiementController.confirmerPaiement() — retourne HTTP 200 avec le nouveau statut.

Gestion terrain de padel

CF-M-004	Un match privé non complet (moins de 4 joueurs ayant payé) la veille (24h avant début) du match bascule automatiquement en public.
	DB : Colonne <code>type_match</code> ENUM (PRIVE, PUBLIC) dans Match. Job planifié (Spring @Scheduled) vérifiant chaque nuit les matchs privés du lendemain. Service : <code>MatchService.basculerMatchesIncomplets()</code> — exécuté chaque nuit, change le <code>type_match</code> à PUBLIC si <code>count(participations confirmées) < 4</code>. Controller : Pas d'endpoint direct — traitement automatique. <code>MatchController.getMatch()</code> retourne le type actuel.
CF-M-005	Si un match privé bascule en public faute de 4 joueurs, l'organisateur reçoit une pénalité de 7 jours l'empêchant de créer un nouveau match.
	DB : Table Penalite (id, date_debut, date_fin, motif, FK matricule). Créée automatiquement lors du basculement. Service : <code>PenaliteService.appliquerPenalite(matricule)</code> — crée une entrée Penalite avec <code>date_fin = date_debut + 7 jours</code>. Controller : <code>MatchController.creerMatch()</code> — appelle <code>PenaliteService.estEnPenalite(matricule)</code> avant création, retourne HTTP 403 si pénalité active.
CF-M-006	Si un joueur n'a pas payé la veille du match (24h avant), sa place est libérée et le match devient public si c'était un match privé.
	DB : Job planifié vérifiant les participations EN_ATTENTE pour les matchs du lendemain. Statut mis à ANNULEE, <code>type_match</code> basculé si nécessaire. Service : <code>MatchService.libererPlacesNonPayees()</code> — exécuté chaque nuit, annule les participations non payées. Controller : Pas d'endpoint direct — traitement automatique.
CF-M-007	Si un match public se termine avec des places non remplies, l'organisateur doit payer le solde (places vides × 15€). Ce solde est ajouté à son prochain paiement.
	DB : Colonne <code>solde_du</code> DECIMAL dans la table Membre. Mise à jour après le match si places non remplies. Service : <code>MatchService.calculerSoldeOrganisateur(matchId)</code> — calcule $(4 - nb_joueurs_confirmes) \times 15$ et met à jour <code>solde_du</code> du membre. Controller : <code>PaiementController.initierPaiement()</code> — inclut automatiquement <code>solde_du</code> dans le montant total.

Gestion terrain de padel

CF-M-008	La pénalité de 7 jours et le solde dû sont cumulables. Un organisateur peut avoir les deux simultanément.
	DB : Les deux sont stockés séparément : table <i>Penalite</i> + colonne <i>solde_du</i> dans <i>Membre</i>. Aucun lien d'exclusion. Service : <i>MatchService.basculerMatchesIncomplets()</i> — appelle <i>PenaliteService.appliquerPenalite()</i> ET <i>MatchService.calculerSoldeOrganisateur()</i> si applicable. Controller : <i>MatchController.creerMatch()</i> — vérifie pénalité ET <i>solde_du</i> avant autorisation.
CF-M-009	Un match privé peut être annulé par l'organisateur jusqu'à 48h avant le début du match. Un match public jusqu'à 24h avant.
	DB : Colonne <i>date_heure_debut</i> dans <i>Disponibilite</i>. Vérification en service avant annulation. Service : <i>MatchService.annulerMatch(matchId, matricule)</i> — vérifie que l'appelant est l'organisateur ET que le délai est respecté. Lève une exception sinon. Controller : <i>MatchController.annulerMatch()</i> — retourne HTTP 403 si délai dépassé ou si non organisateur.
CF-M-010	Dans un match public, l'organisateur ne peut pas ajouter directement d'autres joueurs. Les joueurs s'inscrivent eux-mêmes.
	DB : Pas de contrainte DB spécifique — géré en logique service. Service : <i>MatchService.ajouterJoueur()</i> — vérifie si le match est PUBLIC et si l'appelant est l'organisateur. Si oui, lève une exception. Controller : <i>MatchController.ajouterJoueur()</i> — retourne HTTP 403 si tentative d'ajout direct dans un match public.

Contraintes liées aux Réservations (CF-R)

Code	Description / Implémentation
CF-R-001	Chaque créneau de disponibilité a une durée fixe de 1h30 avec 15 minutes d'intervalle entre deux créneaux consécutifs.
	DB : Colonne <i>date_heure_debut</i> dans <i>Disponibilite</i>. La <i>date_heure_fin</i> est calculée : <i>date_heure_debut</i> + 1h30. Le prochain créneau commence à <i>date_heure_debut</i> + 1h45. Service : <i>DisponibiliteService.genererCreneaux()</i> — boucle avec intervalle de 105 minutes (1h30 + 15min) entre chaque <i>date_heure_debut</i>.

Gestion terrain de padel

	Controller : <i>AdminController.genererCreneaux()</i> — déclenche la génération pour un site et une année donnés.
CF-R-002	Les horaires d'ouverture sont définis par site et valables pour une année civile complète (1er janvier au 31 décembre).
	DB : <i>Table HoraireAnnuel (id, annee, heure_ouverture, heure_fermeture, FK id_site). Contrainte UNIQUE sur (annee, id_site).</i> Service : <i>HoraireService.getHorairePourAnnee(siteld, annee)</i> — retourne les horaires ou lève une exception si non configurés. Controller : <i>AdminController.definirHoraires()</i> — crée ou met à jour les horaires pour un site et une année.
CF-R-003	Les fermetures peuvent être globales (tous sites) ou spécifiques à un site, récurrentes (jour de semaine) ou ponctuelles (date précise). Tous ces cas sont exclus de la génération des créneaux.
	DB : <i>Tables FermetureRecurrente (jour_semaine 0-6, FK id_site nullable) et FermeturePonctuelle (date, motif, FK id_site nullable). FK nullable = fermeture globale.</i> Service : <i>FermetureService.estJourFerme(date, siteld)</i> — vérifie les 4 types de fermeture et retourne true si le jour est fermé. Controller : <i>AdminController.ajouterFermeture()</i> — crée la fermeture selon le type et le scope (global ou site).
CF-R-004	Un créneau déjà réservé ne peut pas être attribué à un second match.
	DB : <i>Colonne statut ENUM (LIBRE, RESERVE) dans Disponibilite. Contrainte : un seul Match peut référencer une Disponibilite (FK unique dans match).</i> Service : <i>DisponibiliteService.verifierDisponibilite(dispond)</i> — vérifie statut = LIBRE avant réservation. Controller : <i>MatchController.creerMatch()</i> — retourne HTTP 409 si le créneau est déjà réservé.

Contraintes liées aux Membres (CF-MB)

Code	Description / Implémentation
CF-MB-001	Le délai de réservation (création de match) dépend du type de membre : Global = 21 jours, Site = 14 jours, Libre = interdit.
	DB : Table <i>TypeMembre</i> (<i>id</i>, <i>libelle</i>, <i>prefixe</i>, <i>delai_reservation_jours</i>). Valeurs : G=21, S=14, L=0 (non applicable). Service : <i>MembreService.verifierDelaiReservation(matricule, dateMatch)</i> — récupère le délai via <i>TypeMembre</i> et vérifie que <i>dateMatch</i> >= <i>today</i> + <i>delai</i>. Controller : <i>MatchController.creerMatch()</i> — appelle <i>verifierDelaiReservation()</i>, retourne HTTP 403 si délai non respecté.
CF-MB-002	Le membre Libre ne peut pas créer de match. Il peut uniquement s'inscrire à des matchs publics existants dans les 5 derniers jours.
	DB : Colonne <i>prefixe</i> dans <i>TypeMembre</i> permet d'identifier le type L. Pas de contrainte DB — géré en service. Service : <i>MatchService.creerMatch()</i> — vérifie le type du membre. Si type = LIBRE, lève une exception métier. Controller : <i>MatchController.creerMatch()</i> — retourne HTTP 403 avec message explicite si membre de type Libre.
CF-MB-003	Un membre en période de pénalité active ne peut pas créer de nouveau match.
	DB : Table <i>Penalite</i> (<i>date_debut</i>, <i>date_fin</i>, FK <i>matricule</i>). Requête : <i>SELECT COUNT(*) WHERE matricule = ? AND date_fin >= NOW()</i>. Service : <i>PenaliteService.estEnPenalite(matricule)</i> — retourne true si une pénalité active existe pour ce matricule. Controller : <i>MatchController.creerMatch()</i> — appelle <i>estEnPenalite()</i> avant création, retourne HTTP 403 avec date de fin de pénalité.
CF-MB-004	Un membre avec un solde dû le voit automatiquement ajouté à son prochain paiement. Le solde est remis à zéro après paiement.
	DB : Colonne <i>solde_du</i> DECIMAL(10,2) DEFAULT 0.00 dans <i>Membre</i>. Service : <i>PaiementService.calculerMontantTotal(matricule)</i> — retourne 15.00 + <i>membre.solde_du</i>. <i>PaiementService.confirmerPaiement()</i> — remet <i>solde_du</i> à 0 après paiement. Controller : <i>PaiementController.initierPaiement()</i> — affiche le détail du montant (15€ part + solde éventuel).

Gestion terrain de padel

CF-MB-005	Le membre de Site ne peut créer un match que sur son site d'appartenance.
	DB : Colonne FK <i>id_site</i> dans <i>Membre</i> (nullable pour G et L, obligatoire pour S). Service : <i>MatchService.verifierAccesSite(matricule, siteld)</i> — si <i>type = SITE</i>, vérifie que <i>membre.id_site = siteld</i>. Controller : <i>MatchController.creerMatch()</i> — retourne HTTP 403 si membre S tente de réserver sur un autre site.
CF-MB-006	Un membre peut avoir deux matricules simultanément (G + S) en créant deux comptes distincts.
	DB : Unicité email assurée dans la table <i>personne</i> — un email peut avoir plusieurs matricules. Service : Aucune validation spécifique — comportement permis par conception. Controller : <i>AuthController.inscription()</i> — ne bloque pas les inscriptions multiples avec le même email.
CF-MB-007	Règle de cumul des comptes
	DB : Le trigger vérifie trois règles : (1) une <i>Personne</i> ne peut pas avoir deux <i>Membres</i> de type G, (2) une <i>Personne</i> ne peut pas avoir deux <i>Membres</i> de type S, (3) un <i>Membre</i> de type L ne peut pas coexister avec un <i>Membre</i> G ou S pour la même <i>Personne</i>. Service : <i>MembreService.verifierCumulAutorise(id_personne, type_demande)</i>. Controller : <i>AuthController.inscription()</i> → HTTP 403 si cumul interdit.
CF-MB-008	Blocage croisé G + S
	DB : Les vérifications de pénalité et solde se font via <i>id_personne</i>. Service : <i>PenaliteService.estEnPenalite(id_personne)</i>, <i>PaiementService.remettreASoldeZero(id_personne)</i>. Controller : <i>MatchController.creerMatch()</i> vérifie via <i>id_personne</i>.
CF-MB-009	Génération automatique du matricule
	DB : Séquence PostgreSQL séparée par type (<i>seq_global</i>, <i>seq_site</i>, <i>seq_libre</i>). Service : <i>MembreService.genererMatricule(typeld)</i> — récupère le préfixe dans <i>TypeMembre</i> et incrémente la séquence correspondante. Controller : <i>AuthController.inscription()</i> — appelle <i>genererMatricule()</i> avant l'INSERT."

Contraintes liées aux Sites et Terrains (CF-S)

Code	Description / Implémentation
CF-S-001	Seul l'administrateur global peut créer un nouveau site.
	DB : Pas de contrainte DB — géré par le système d'autorisation JWT. Service : <code>SiteService.creerSite()</code> — vérifie que l'appelant a le rôle ADMIN_GLOBAL. Controller : <code>SiteController.creerSite()</code> — annoté <code>@PreAuthorize("hasRole('ADMIN_GLOBAL')")</code>, retourne HTTP 403 sinon.
CF-S-002	L'administrateur global et l'administrateur de site peuvent tous les deux gérer les terrains d'un site. L'admin de site ne peut gérer que les terrains de son propre site.
	DB : Table Terrain (id, numero, statut, FK id_site). Colonne statut ENUM (ACTIF, INACTIF). Service : <code>TerrainService.gererTerrain()</code> — vérifie que si rôle = ADMIN_SITE, alors <code>terrain.id_site = admin.id_site</code>. Controller : <code>TerrainController</code> — méthodes accessibles aux deux rôles admin, avec vérification du scope en service.
CF-S-003	La génération des créneaux doit être relancée après toute modification des horaires ou des fermetures. Les créneaux déjà réservés ne sont jamais supprimés lors d'une régénération.
	DB : Lors de la régénération, seules les Disponibilités avec statut = LIBRE sont supprimées avant recréation. Service : <code>DisponibiliteService.regenererCreneaux(siteId, annee)</code> — supprime les LIBRE, recrée tous les créneaux selon les nouveaux horaires et fermetures. Controller : <code>AdminController.genererCreneaux()</code> — retourne un résumé (nb créés, nb conservés réservés, nb supprimés).

Contraintes de Sécurité (CF-SEC)

Code	Description / Implémentation
------	------------------------------

Gestion terrain de padel

CF-SEC-001	L'authentification se fait via matricule + mot de passe. Le système génère un JWT token contenant le rôle. Toutes les routes API sont protégées sauf l'inscription et la connexion.
	DB : Table Membre avec colonne <code>mot_de_passe</code> (hashé BCrypt). Pas de token stocké en DB — <i>stateless</i> JWT. Service : <code>AuthService.authentifier(matricule, mdp)</code> — vérifie les credentials, génère le JWT via <code>JwtService</code>. Controller : <code>AuthController.login()</code> — endpoint public. Tous les autres controllers annotés <code>@PreAuthorize</code> selon le rôle requis.
CF-SEC-002	La base de données utilise des utilisateurs SQL avec des droits spécifiques. Aucun utilisateur applicatif n'a les droits superuser.
	DB : User <code>app_user</code> : <code>SELECT, INSERT, UPDATE, DELETE</code> sur les tables applicatives. User <code>app_readonly</code> : <code>SELECT</code> uniquement (pour les stats). Aucun <code>GRANT ALL</code>. Service : Configuration dans <code>application.properties</code> : <code>spring.datasource.username=app_user</code>. Controller : Pas d'impact direct — la sécurité est au niveau de la connexion à la DB.

Description des entités

Cette section décrit les 15 entités qui composent la base de données de l'application Padel Manager. Pour chaque entité, les attributs sont détaillés avec leur type, leur rôle de clé (**PK** = clé primaire, **FK** = clé étrangère) et leur description.

PK — Clé primaire (identifiant unique)	FK — Clé étrangère (référence)	SERIAL — Auto-incrémenté PostgreSQL
---	---------------------------------------	--

Personne

Personne			
<i>Représente une personne physique unique. Séparée de Membre parce qu'une même personne peut avoir jusqu'à deux matricules (G+S). C'est au niveau de la Personne que se font les vérifications de blocage croisé.</i> <i>Pénalité et solde bloquent les deux comptes simultanément.</i>			
Attribut	Type	Clé	Description
id_personne	SERIAL	PK	Identifiant unique auto-incrémenté
nom	VARCHAR(100)		Nom de famille
prenom	VARCHAR(100)		Prénom
email	VARCHAR(255)	UK	Adresse email unique, une personne physique = une seule ligne
téléphone	VARCHAR(20)		Numéro de téléphone (optionnel)

Remarque: email UNIQUE dans personne. À l'inscription, si l'email existe déjà, le système lie le nouveau Membre à la Personne existante plutôt que d'en créer une nouvelle.

TypeMembre

TypeMembre			
<i>Définit les trois types de membres avec leurs règles de délai de réservation. Entité configurable évitant de coder les règles en dur dans l'application.</i>			
Attribut	Type	Clé	Description
id_type	SERIAL	PK	Identifiant unique
libelle	VARCHAR(50)		Nom du type : Global, Site, Libre
prefixe	CHAR(1)	UK	Préfixe du matricule : G, S ou L

Gestion terrain de padel

delai_reservation_jours	INTEGER		Délai avant lequel un match peut être créé (21 pour G, 14 pour S, 0 pour L — non applicable)
peut_creer_match	BOOLEAN		Indique si ce type peut créer un match (false pour Libre)

Membre

Membre			
<i>Représente le rôle métier d'une personne dans le système. Contient le matricule, les informations d'authentification et l'état financier du membre.</i>			
Attribut	Type	Clé	Description
matricule	VARCHAR(10)	PK	Identifiant unique du membre. Format : Gxxxx / Sxxxx / Lxxxx. Généré automatiquement à l'inscription.
mot_de_passe	VARCHAR(255)		Mot de passe hashé avec BCrypt
date_inscription	DATE		Date de création du compte
solde_du	DECIMAL(10,2)		Solde dû par l'organisateur suite à des places non remplies. Ajouté automatiquement au prochain paiement. Défaut : 0.00
id_personne	INTEGER	FK	Référence vers la personne physique Permet les blocage croisé G + S via id_personne
id_type	INTEGER	FK	Référence vers le type de membre (table TypeMembre)
id_site	INTEGER	FK nullable	Référence vers le site d'appartenance. NULL pour les types Global et Libre, obligatoire pour le type Site.

Remarque: Une personne peut avoir au maximum un Membre de type G et un Membre de type S. Le type L est exclusif.

Il ne peut pas coexister avec G ou S. Vérifié par trigger à l'insertion.

Penalite

Penalite			
<i>Enregistre les pénalités appliquées à un organisateur dont le match privé n'a pas atteint 4 joueurs. Pendant la période de pénalité, le membre ne peut pas créer de nouveau match.</i>			
Attribut	Type	Clé	Description
id_penalite	SERIAL	PK	Identifiant unique
date_debut	TIMESTAMP		Date et heure de début de la pénalité
date_fin	TIMESTAMP		Date et heure de fin de la pénalité (date_debut + 7 jours)
motif	VARCHAR(255)		Description du motif (ex : Match privé #40 incomplet — UC-03)
matricule	VARCHAR(10)	FK	Référence vers le membre pénalisé (table Membre)

Remarque: Utilisation de matricule et non id_personne, la pénalité est liée à l'acte de l'organisateur avec un matricule précis.

La vérification de blocage remonte à id_perosnne pour toucher les 2 comptes.

Le service fait: **SELECT p FROM Penalite p JOIN Membre m ON p.matricule = m.matricule WHERE m.id_personne = ? AND p.date_fin >= NOW().**

Administrateur

Administrateur			
<i>Représente les comptes administrateurs du système. Séparé des membres car les admins ont un accès différent et sont créés directement en base par l'admin global. Pas d'inscription publique.</i>			
Attribut	Type	Clé	Description
id_admin	SERIAL	PK	Identifiant unique
nom	VARCHAR(100)		Nom de l'administrateur
prenom	VARCHAR(100)		Prénom de l'administrateur
email	VARCHAR(255)	UK	Email unique de l'administrateur

Gestion terrain de padel

mot_de_passe	VARCHAR(255)		Mot de passe hashé avec BCrypt
role	VARCHAR(20)		Rôle : ADMIN_GLOBAL ou ADMIN_SITE
id_site	INTEGER	FK nullable	Référence vers le site géré. NULL pour ADMIN_GLOBAL, obligatoire pour ADMIN_SITE.

Site

Site			
<i>Représente un site physique de padel. Chaque site peut avoir un nombre variable de terrains, des horaires d'ouverture propres et des jours de fermeture spécifiques.</i>			
Attribut	Type	Clé	Description
id_site	SERIAL	PK	Identifiant unique
nom	VARCHAR(150)		Nom du site (ex : Padel Brussels Center)
adresse	VARCHAR(255)		Adresse complète du site
ville	VARCHAR(100)		Ville du site
actif	BOOLEAN		Indique si le site est actif. Défaut : true.

Terrain

Terrain			
<i>Représente un terrain de padel appartenant à un site. Les créneaux de disponibilité sont générés par terrain.</i>			
Attribut	Type	Clé	Description
id_terrain	SERIAL	PK	Identifiant unique
numero	INTEGER		Numéro du terrain au sein du site (ex : 1, 2, 3)

Gestion terrain de padel

statut	VARCHAR(10)		ACTIF ou INACTIF. Un terrain inactif n'apparaît pas dans les créneaux disponibles.
id_site	INTEGER	FK	Référence vers le site d'appartenance (table Site)

HoraireAnnuel

HoraireAnnuel

Définit les horaires d'ouverture d'un site pour une année civile complète. Ces horaires servent de base à la génération des créneaux de disponibilité.

Attribut	Type	Clé	Description
id_horaire	SERIAL	PK	Identifiant unique
annee	INTEGER		Année civile concernée (ex : 2025, 2026)
heure_ouverture	TIME		Heure d'ouverture du site (ex : 09:00)
heure_fermeture	TIME		Heure de fermeture du site (ex : 22:00)
id_site	INTEGER	FK	Référence vers le site concerné. Contrainte UNIQUE sur (annee, id_site).

Remarque: UNIQUE sur (année,id_site). Un seul horaire par site par année

FermetureRecurrente

FermetureRecurrente

Définit un jour de fermeture hebdomadaire récurrent pour un site ou pour tous les sites. Ex : fermé tous les lundis. FK id_site nullable : NULL = fermeture globale tous sites.

Attribut	Type	Clé	Description
id_fermeture_rec	SERIAL	PK	Identifiant unique
jour_semaine	INTEGER		Jour de la semaine : 0 = lundi, 1 = mardi, ..., 6 = dimanche
motif	VARCHAR(255)		Motif de la fermeture (optionnel, ex : jour de repos)

Gestion terrain de padel

id_site	INTEGER	FK nullable	Référence vers le site concerné. NULL si fermeture globale (tous les sites).
----------------	---------	------------------------	--

FermeturePonctuelle

FermeturePonctuelle			
<i>Définit une fermeture exceptionnelle à une date précise pour un site ou pour tous les sites. Ex : 1er janvier, travaux. FK id_site nullable : NULL = fermeture globale tous sites.</i>			
Attribut	Type	Clé	Description
id_fermeture_ponc	SERIAL	PK	Identifiant unique
date_fermeture	DATE		Date précise de la fermeture
motif	VARCHAR(255)		Motif de la fermeture (ex : Jour férié national, Travaux)
id_site	INTEGER	FK nullable	Référence vers le site concerné. NULL si fermeture globale (tous les sites).

Remarque: Une fermeture récurrente n'a pas de date, elle a un jour de semaine. Une fermeture ponctuelle n'a pas de jour de semaine, elle a une date.

Les fusionner aurait créé des colonnes NULL selon le type.

Disponibilite

Disponibilite			
<i>Représente un créneau de jeu pré-généré pour un terrain donné. Chaque créneau dure 1h30. Les créneaux sont générés à partir des HoraireAnnuel en excluant les jours de fermeture.</i>			
Attribut	Type	Clé	Description
id_dispo	SERIAL	PK	Identifiant unique
date_heure_debut	TIMESTAMP		Date et heure de début du créneau
date_heure_fin	TIMESTAMP		Date et heure de fin du créneau (= date_heure_debut + 1h30)

Gestion terrain de padel

statut	VARCHAR(10)		LIBRE ou RESERVE. Mis à RESERVE lorsqu'un match occupe ce créneau.
id_terrain	INTEGER	FK	Référence vers le terrain concerné (table Terrain)

Remarque: UNIQUE sur (date_heure_debut, id_terrain).

Impossible d'avoir deux créneaux au même moment sur le même terrain.

MatchPadel

MatchPadel			
<i>Représente un match de padel occupant un créneau disponible. Un match est privé (organisateur gère les joueurs) ou public (places ouvertes à tous les membres éligibles).</i>			
Attribut	Type	Clé	Description
id_match	SERIAL	PK	Identifiant unique
type_match	VARCHAR(10)		PRIVE ou PUBLIC
statut	VARCHAR(20)		EN_ATTENTE, OUVERT, COMPLET, ANNULE, TERMINE
montant_total	DECIMAL(10, 2)		Montant total du match. Défaut : 60.00€
date_creation	TIMESTAMP		Date et heure de création du match
id_dispo	INTEGER	FK UK	Référence vers le créneau occupé (table Disponibilite). Contrainte UNIQUE.
matricule_organisateur	VARCHAR(10)	FK	Référence vers le matricule de l'organisateur (table Membre)

Participation

Participation			
<i>Entité d'association entre un Membre et un Match. Représente l'inscription d'un joueur à un match. Un match peut avoir au maximum 4 participations.</i>			
Attribut	Type	Clé	Description
id_participation	SERIAL	PK	Identifiant unique
statut	VARCHAR(15)		EN_ATTENTE, CONFIRMEE, ANNULEE. Passe à CONFIRMEE après paiement.
date_inscription	TIMESTAMP		Date et heure d'inscription au match
id_match	INTEGER	FK	Référence vers le match concerné (table Match)
matricule	VARCHAR(10)	FK	Référence vers le membre participant (table Membre)

Paieement

Paieement			
<i>Enregistre le paiement effectué par un joueur pour confirmer sa participation. Lié à une Participation, pas directement au Match. Le montant inclut la part du match (15€) plus le solde dû éventuel.</i>			
Attribut	Type	Clé	Description
id_paiement	SERIAL	PK	Identifiant unique
montant	DECIMAL(10,2)		Montant payé (minimum 15€, peut inclure un solde dû)
date_paiement	TIMESTAMP		Date et heure du paiement
statut	VARCHAR(15)		EN_ATTENTE ou CONFIRME
solde_inclus	DECIMAL(10,2)		Montant du solde dû inclus dans ce paiement. Défaut : 0.00
id_participation	INTEGER	FK UK	Référence vers la participation concernée (table Participation). Contrainte UNIQUE.

RefreshToken

Participation			
<i>Entité d'association entre un Membre et un Match. Représente l'inscription d'un joueur à un match. Un match peut avoir au maximum 4 participations.</i>			
Attribut	Type	Clé	Description
id_refresh_token	BIGSERIAL	PK	Identifiant unique auto-incrémenté
token	VARCHAR(255)		Valeur du refresh token (UUID généré aléatoirement). UNIQUE et NOT NULL.
date_creation	TIMESTAMP		Date et heure de création du token. Défaut : NOW().
date_expiration	TIMESTAMP		Date et heure d'expiration du token. Passé ce délai, le token est rejeté même s'il n'est pas révoqué.
revoque	BOOLEAN		Indique si le token a été explicitement révoqué (ex : déconnexion). Défaut : false.
matricule	VARCHAR(10)	FK	Référence vers le membre participant (table Membre)

Remarque : Deux index sont créés sur cette table pour des raisons de performance : `idx_refresh_token_matricule` (recherche par membre) et `idx_refresh_token_token` (vérification rapide du token à chaque requête de renouvellement). La révocation se fait via `UPDATE revoque = true` — aucune suppression physique immédiate, ce qui permet d'auditer les sessions. Un job de nettoyage peut purger les tokens expirés et révoqués périodiquement.

Schéma relationnel

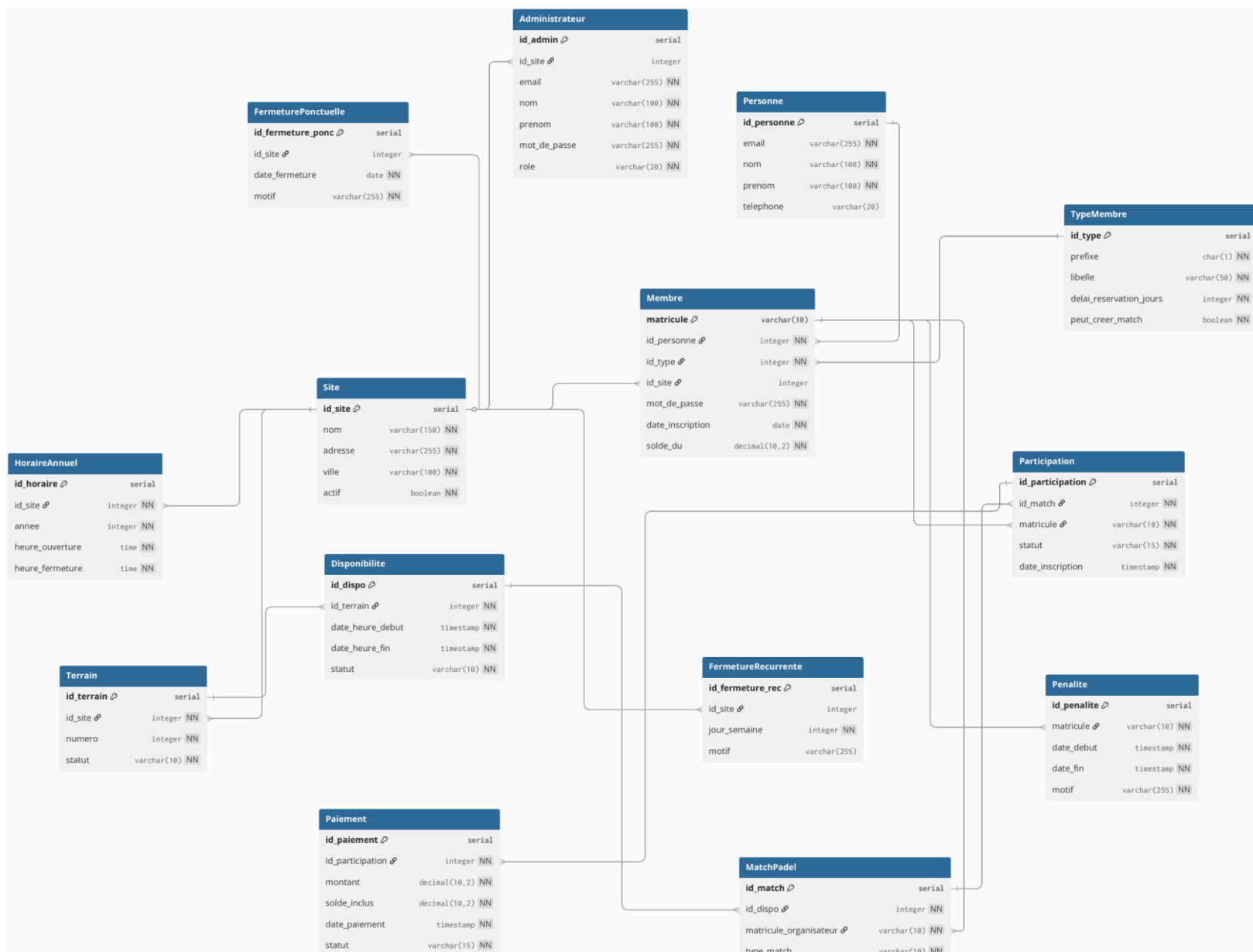
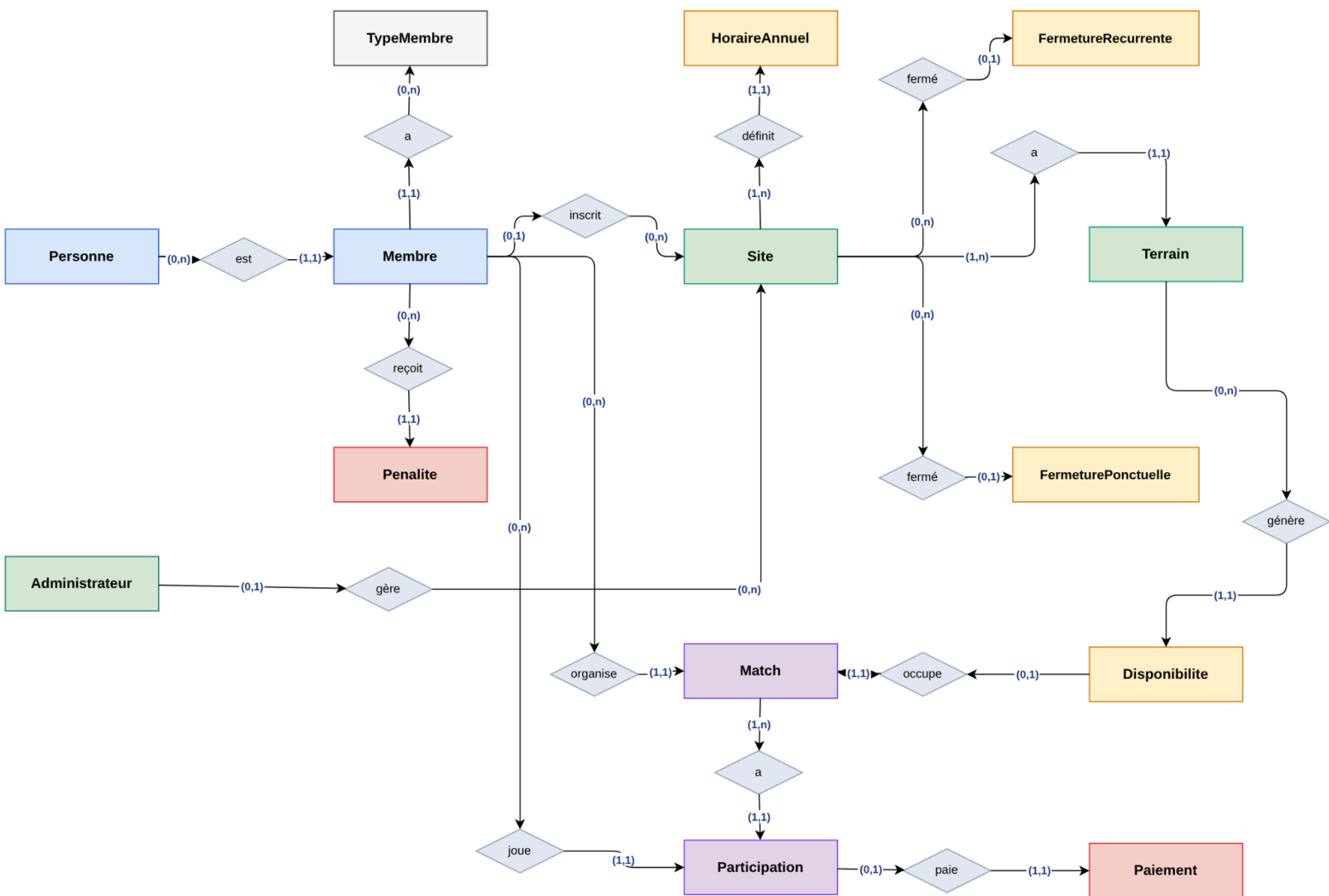


Schéma EA



Analyse technique

Stack technologique

L'ensemble des technologies utilisées dans ce projet sont soit imposées par les cours PDW et SGBD, soit choisies pour leur cohérence avec les contraintes du projet et la facilité de déploiement.

Couche	Technologie	Version	Justification
Frontend	Angular	20+	Imposé par le cours PDW
Backend	Java + Spring Boot	21+ / 3.5+	Imposé par le cours PDW
Base de données	PostgreSQL	16+	Relationnelle, support natif TIME/TIMESTAMP/SERIAL
Conteneurisation	Docker + Docker Compose	—	Démarrage en une commande, aucune installation manuelle
Authentification	JWT (JSON Web Token)	—	Stateless, gestion des rôles, voir section 5
Migration BDD	Flyway	—	Seed et schéma automatiques au premier démarrage
Documentation API	SpringDoc OpenAPI (Swagger)	3+	Génération automatique, accessible via /swagger-ui
Build backend	Maven	3.9+	Gestion des dépendances Java
Build frontend	npm + Angular CLI	—	Gestion des dépendances Angular

Architecture Backend - Spring Boot

Le backend respecte une architecture en couches. Chaque couche a une responsabilité unique et ne communique qu'avec la couche directement en dessous.



Controllers — @RestController

Points d'entrée de l'API REST. Reçoivent les requêtes HTTP, appellent le service correspondant, retournent les réponses avec les codes HTTP appropriés (200, 201, 400, 403, 404, 409). La sécurité par rôle est appliquée via @PreAuthorize.

Services — @Service

Couche métier. Contiennent toutes les règles de gestion : vérification des fenêtres de réservation, validation des pénalités, calcul des soldes, logique de bascule privé→public. C'est ici que vivent les règles CF-M, CF-R, CF-MB et CF-S.

Repositories — JpaRepository

Interfaces héritant de JpaRepository<Entité, ID>. Spring Data JPA génère automatiquement les requêtes SQL pour les opérations CRUD. Les requêtes complexes (ex : vérifier une pénalité active via id_personne) sont définies avec @Query.

Models — @Entity

Classes Java annotées @Entity représentant les 15 tables de la base de données. Les relations sont mappées avec @ManyToOne, @OneToMany, @OneToOne selon le schéma relationnel.

DTOs — Data Transfer Objects

Objets de transfert entre le controller et le service. Évitent d'exposer directement les entités JPA à l'extérieur. Un DTO par opération significative (ex : MatchCreationDTO, PaiementDTO, MembreResponseDTO).

Injection de dépendances

Assurée par Spring via injection par constructeur sur tous les services et controllers. Aucun new dans le code applicatif — le conteneur Spring gère le cycle de vie de tous les composants.

Architecture Frontend - Angular

L'application Angular est structurée par feature modules. Chaque grande fonctionnalité est isolée dans son propre dossier. Une seule application Angular est développée avec des vues différentes selon le rôle.

```
src/
├── app/
│   ├── core/      → Services singleton, intercepteurs HTTP, guards
│   │   ├── auth/  → AuthService, JwtInterceptor
│   │   │   └── guards/ → AuthGuard, RoleGuard
│   │   ├── shared/ → Composants réutilisables, pipes, modèles TypeScript
│   │   └── features/
│   │       ├── auth/ → Login, inscription
│   │       ├── membre/ → Tableau de bord, matches publics, réservations
│   │       └── admin/ → Dashboard admin, gestion sites, statistiques
│   └── app-routing.module.ts
```

La navigation est filtrée par des Route Guards (CanActivate) qui lisent le rôle contenu dans le JWT token. Un membre ne peut pas accéder aux routes /admin et un admin de site ne peut pas accéder aux données d'un autre site.

La communication avec le backend se fait exclusivement via HttpClient Angular. Un JwtInterceptor ajoute automatiquement le token Authorization à chaque requête sortante. Le frontend ne connaît pas l'existence de PostgreSQL.

Base de données - PostgreSQL

Justification du choix PostgreSQL

PostgreSQL a été retenu pour les raisons suivantes : support natif des types TIME, TIMESTAMP, DECIMAL et SERIAL utilisés dans notre schéma ; robustesse et fiabilité éprouvée ; entièrement open source ; compatible Docker sans configuration supplémentaire ; recommandé par le cours SGBD.

Démarrage via Docker Compose

Aucune installation manuelle de PostgreSQL n'est requise. Un docker compose up suffit pour démarrer la base de données.

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: padel_manager
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: padel_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

Initialisation automatique — Flyway

Le schéma et les données de test sont créés automatiquement au premier démarrage via Flyway, intégré à Spring Boot. Aucun script SQL manuel à exécuter.

- ➔ V1_create_schema.sql — création des 14 tables avec toutes les contraintes, indexes et triggers.
- ➔ V2_create_users.sql — Création des utilisateurs SQL app_user et app_readonly avec leurs droits respectifs (CF-SEC-002).

- V3_seed_data.sql — insertion des données de test : 2 sites, 3 terrains par site, 3 types de membres, membres et admins de test.
- V4_grant_permissions.sql — Attribution des droits aux utilisateurs SQL : SELECT / INSERT / UPDATE / DELETE pour app_user, SELECT uniquement pour app_readonly.
- V5_refresh_token.sql — Ajout de la table refresh_token et de ses index. Migration séparée car ajoutée après la mise en place du schéma initial.

Exigence du cours PDW : aucun logiciel à installer à part Docker. Aucun script SQL ne sera exécuté manuellement. Ces deux conditions sont respectées.

Sécurité — Utilisateurs SQL

Conformément à la contrainte CF-SEC-002, la base de données utilise deux utilisateurs SQL aux droits distincts. Aucun utilisateur applicatif ne dispose des droits superuser.

Utilisateur	Droits	Usage
app_user	SELECT, INSERT, UPDATE, DELETE sur les tables applicatives	Connexion applicative Spring Boot (opérations CRUD)
app_readonly	SELECT uniquement sur toutes les tables	Requêtes de statistiques et reporting admin

Configuration dans Spring Boot :

```
# application.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/padel_manager
spring.datasource.username=app_user
spring.datasource.password=padel_password
```

Choix d'accès aux données

L'application accède directement aux tables via Spring Data JPA, sans passer par des vues ou des procédures stockées. Ce choix simplifie le développement et la maintenance pour un projet solo.

La contrepartie de ce choix est que la logique métier doit être entièrement gérée dans la couche service Spring Boot — ce qui est le cas dans notre architecture. Les règles ne sont jamais dispersées entre le SQL et le Java.

Authentification — JWT

L'authentification se fait via matricule + mot de passe, stockés en base de données. Ce choix satisfait les deux cours simultanément :

- Cours SGBD : "pas de login, uniquement le matricule" — interprété comme : le matricule est l'identifiant principal
- Cours PDW : mécanisme d'authentification avec gestion des rôles — satisfait par JWT

Flux d'authentification

1. Angular envoie POST /api/auth/login
Body: { matricule: "G0001", mot_de_passe: "..." }
2. Spring Boot vérifie les credentials en BDD
(BCrypt.matches pour comparer les mots de passe hashés)
3. Spring Boot génère deux tokens signés :
 - access_token : JWT court (ex : 15 min) — { matricule, role, id_personne, exp }
 - refresh_token : UUID opaque, long (ex : 7 jours), stocké en BDD (table refresh_token)
4. Angular stocke les deux tokens en mémoire.
Chaque requête protégée envoie : Authorization: Bearer <access_token>
5. Spring Security valide l'access token à chaque requête protégée
6. Quand l'access token expire, Angular envoie POST /api/auth/refresh
Body: { refresh_token: "<uuid>" }
Spring Boot vérifie : token présent en BDD + non révoqué + non expiré
→ Retourne un nouvel access_token (et optionnellement un nouveau refresh_token)
7. Déconnexion : POST /api/auth/logout
→ UPDATE refresh_token SET revoke = true WHERE token = ?

Remarque : Ce mécanisme évite de demander à l'utilisateur de se reconnecter à chaque expiration de l'access token, tout en permettant une révocation explicite (logout). L'access token reste stateless (non stocké en BDD) ; seul le refresh token est persisté.

Sécurité des mots de passe

Les mots de passe sont hashés avec BCrypt avant stockage en base de données. Ils ne sont jamais stockés en clair. Le sel est généré aléatoirement par BCrypt à chaque hash.

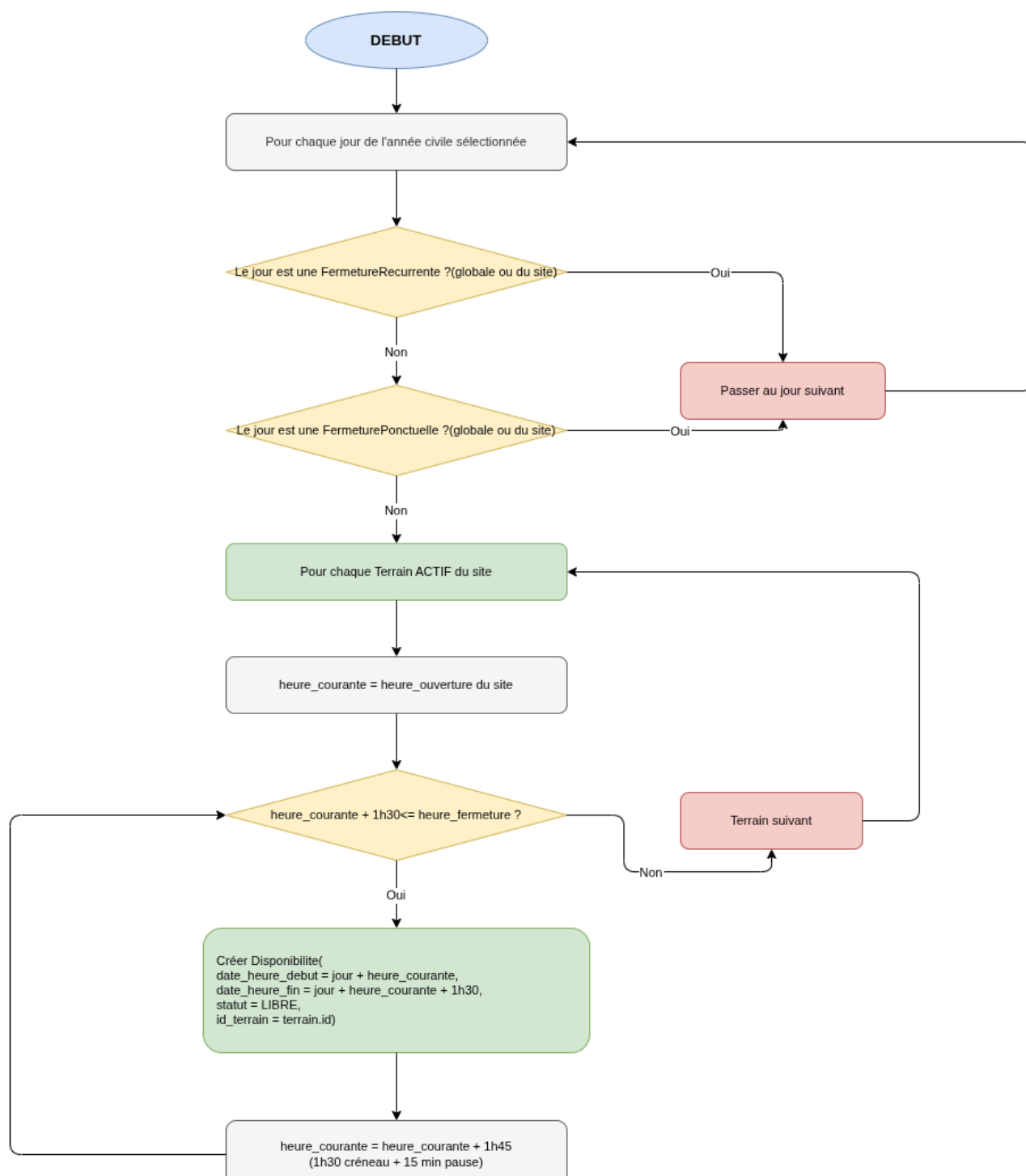
Les routes publiques (inscription, login) ne nécessitent pas de token. Toutes les autres routes sont protégées par Spring Security et annotées @PreAuthorize selon le rôle requis.

Génération des Créneaux — Approche Précalculée

Les créneaux de disponibilité sont générés à l'avance pour un site et une année civile donnés, et stockés dans la table Disponibilite. Ce choix a été retenu face à l'approche dynamique.

Critère	Approche précalculée (choisie)	Approche dynamique
Requête de dispo	Simple SELECT WHERE statut = LIBRE	Calcul complexe à chaque requête
Complexité code	Algorithme écrit et testé une seule fois	Logique répliquée à chaque appel
Modification horaires	Régénération nécessaire (cas rare)	Prise en compte automatique
Performance	Excellente (lecture directe)	Variable selon la charge
Adapté solo	Oui	Non — trop complexe

Algorithme de génération



La génération est déclenchée manuellement par un administrateur (global ou de site) via l'interface admin. Elle doit être relancée après toute modification des horaires ou des fermetures. Les créneaux déjà réservés (statut RESERVE) ne sont jamais supprimés lors d'une régénération.

Traitements Automatiques — Spring Scheduler

Deux jobs sont exécutés automatiquement via @Scheduled de Spring Boot. Les deux jobs s'exécutent en se basant sur une fenêtre de 24h avant le match, et non à une heure fixe.

Job 1 — Bascule des matches privés incomplets

Déclenchement : toutes les heures, vérifie les matches dont la date est dans moins de 24h.

Logique : pour chaque match PRIVE dont `date_heure_debut <= NOW() + 24h` et qui a moins de 4 participations CONFIRMÉES :

- Le match passe de PRIVE à PUBLIC (`type_match = PUBLIC`)
- Une Penalite est créée pour l'organisateur avec `date_fin = NOW() + 7 jours`
- Le match devient visible par tous les membres éligibles

Job 2 — Libération des places non payées

Déclenchement : toutes les heures, vérifie les participations dont le match est dans moins de 24h.

Logique : pour chaque Participation EN_ATTENTE dont le Match associé a `date_heure_debut <= NOW() + 24h` :

- La participation passe à ANNULÉE
- Le créneau reste RESERVE mais la place est libérable pour un autre joueur
- Si c'était un match PRIVE avec moins de 4 joueurs restants, le Job 1 prend le relai

Tests

Conformément aux recommandations du cours SGBD, les tests sont réalisés en priorité au niveau de la couche service. Si une règle métier est couverte par un test de service, elle n'est pas redoublée au niveau du controller.

Tests unitaires — Mockito

Testent chaque règle métier en isolation. Les repositories sont mockés via Mockito — aucune base de données réelle n'est requise pour ces tests.

- Vérification de la fenêtre de réservation selon le type de membre (G=21j, S=14j, L=interdit)
- Blocage si pénalité active (vérification via id_personne pour les comptes G+S)
- Calcul du solde dû (places vides x 15€)
- Règles de bascule privé→public
- Validation des contraintes de cumul de comptes (G+S autorisé, L exclusif)

Tests d'intégration — Testcontainers

Testent les repositories avec une vraie instance PostgreSQL lancée dans Docker. Vérifient les contraintes de la base de données que Mockito ne peut pas tester.

- Trigger de limite à 4 participations par match
- Contrainte UNIQUE sur les créneaux (date_heure_debut, id_terrain)
- Contrainte UNIQUE sur id_dispo dans Match (un créneau = un seul match)
- Seed reproductible : chaque test repart d'un état connu

Type	Framework	Couche testée	BDD requise
Tests unitaires	JUnit 5 + Mockito	Service (priorité)	Non — mocks
Tests intégration	Spring Boot Test + Testcontainers	Repository	Oui — PostgreSQL Docker
Tests API (bonus)	MockMvc (@WebMvcTest)	Controller	Non — service mocké

Les tests sont exécutables via mvn test et tournent automatiquement dans le pipeline Git à chaque push.

Résumé des Choix Techniques

Décision	Choix retenu	Raison
Disponibilités	Table précalculée	Plus simple, requêtes rapides, adapté solo
Authentification	JWT maison	Indépendance, pas de service externe requis
Accès données	JPA direct sur tables	Simplicité, logique centralisée en Java
Démarrage BDD	Docker + Flyway	Respect exigence PDW : aucun script manuel
Tests prioritaires	Couche Service	Recommandation prof SGBD
Interface	Single App Angular	Plus simple à maintenir, standard Angular

Annexes

Enoncé Examen